A Comprehensive QA Guide to WCAG 2.2 Level A & AA Standards

Section 1: Introduction to WCAG 2.2 for QA Professionals

1.1 The Importance of Web Accessibility in Quality Assurance

Web accessibility is a critical dimension of overall product quality, extending beyond mere compliance to encompass usability for the broadest possible audience, including individuals with disabilities. For Quality Assurance (QA) professionals, integrating accessibility testing into their workflows is paramount. It ensures that digital products—websites, applications, and other online content—do not erect barriers that prevent people from interacting with or understanding them. This commitment directly influences user experience, expands market reach, enhances brand reputation, and fulfills legal and ethical obligations.[1] Conformance with standards like the Web Content Accessibility Guidelines (WCAG) not only mitigates legal risks but also signals a brand's commitment to inclusivity, which can foster customer loyalty.[1]

The benefits of accessibility often extend to all users, not just those with disabilities. For instance, clear layouts, readable text, and straightforward navigation—hallmarks of accessible design—improve the experience for everyone. Similarly, practices like providing text alternatives for images can enhance Search Engine Optimization (SEO), making content more discoverable.[3] Thus, QA's role in verifying accessibility contributes to a higher-quality, more robust, and more user-friendly product for the entire user base.

Conversely, neglecting accessibility in the QA process can expose an organization to significant risks. Legal repercussions are a primary concern, as many jurisdictions have laws mandating digital accessibility.[1] Beyond legal liabilities, inaccessible products can damage a brand's image, leading to a perception of exclusion or disregard for a segment of the population.[1] This can result in lost revenue, diminished customer trust, and a reduced market share. Therefore, thorough accessibility testing by QA is not just a technical function but a crucial business imperative. Ethical software development inherently includes making products accessible, and QA professionals are key stewards of this principle.[4]

1.2 Overview of WCAG 2.2: Principles (POUR) and Conformance Levels (A, AA)

The Web Content Accessibility Guidelines (WCAG) 2.2, developed by the World Wide

Web Consortium (W3C), provide a global standard for web accessibility. [5] These guidelines are structured around four fundamental principles, often remembered by the acronym POUR, which provide the foundational goals that authors and testers should work toward [5]:

1. **Perceivable:** Information and user interface components must be presentable to users in ways they can perceive. This means users must be able to process the information presented, not that it must be perceivable by all senses. For QA, this involves verifying that alternatives are provided for non-text content, that content is distinguishable (e.g., sufficient color contrast, resizable text), and that content is adaptable to different presentations without loss of information.
2. **Operable:** User interface components and navigation must be operable. Users must be able to interact with all controls and interactive elements. From a QA perspective, this principle covers keyboard accessibility, providing users enough time to read and use content, avoiding content that can cause seizures, and ensuring content is navigable.
3. **Understandable:** Information and the operation of the user interface must be understandable. Users must be able to comprehend the content and the operation of the interface. QA testing for this principle involves checking for readable text, predictable web page operation, and input assistance to help users avoid and correct mistakes.
4. **Robust:** Content must be robust enough that it can be interpreted reliably by a wide variety of user agents, including current and future assistive technologies. For QA, this means verifying that content is compatible with various assistive technologies and that it uses standard technologies appropriately.

Under these principles are 13 guidelines that provide the basic goals. [5] Each guideline has testable **Success Criteria (SC)**. These SCs are assigned one of three levels of conformance [5]:

- **Level A:** This is the minimum level of conformance. Meeting these criteria addresses the most basic web accessibility barriers. Failure to meet Level A criteria means that some groups of users will find it impossible to access the content.
- **Level AA:** This level addresses more significant and common barriers for users with disabilities. Level AA is the conformance level most frequently cited in accessibility policies and legal standards worldwide. It is the generally recommended target for most websites and applications to ensure a good level of accessibility. [8]

- **Level AAA:** This is the highest level of conformance and aims to make content accessible to the widest possible range of users. While desirable, it is not always possible to satisfy all Level AAA SCs for all types of content.

This guide focuses on Level A and Level AA success criteria as they represent the most common and legally mandated targets for accessibility conformance. The POUR principles offer QA professionals a valuable cognitive framework for understanding the *intent* behind each success criterion, moving beyond a simple checklist approach to a more holistic and effective testing methodology. For example, understanding that a criterion falls under "Operable" helps a tester prioritize checks related to keyboard interaction and navigation, while a "Perceivable" criterion would direct attention to aspects like alternative text and color contrast. This principled approach enhances the intuitiveness and thoroughness of accessibility testing.

1.3 Key Changes in WCAG 2.2 (New SCs, Removal of 4.1.1 Parsing)

WCAG 2.2, published on October 5, 2023 [10], builds upon previous versions (2.0 and 2.1) by adding new success criteria and removing one that has become obsolete. It's important for QA professionals to be aware of these changes to ensure their testing methodologies align with the latest standards. WCAG 2.2 is backwards compatible, meaning content that conforms to WCAG 2.2 also conforms to WCAG 2.1 and WCAG 2.0.[6]

New Success Criteria in WCAG 2.2 (Levels A and AA):

WCAG 2.2 introduces nine new success criteria in total. Those relevant to Level A and AA conformance, which are the focus of this guide, include [5]:

- **2.4.11 Focus Not Obscured (Minimum) (Level AA):** Ensures that when a user interface component receives keyboard focus, it is at least partially visible and not entirely hidden by other author-created content (e.g., sticky headers/footers, cookie banners). [5]
- **2.5.7 Dragging Movements (Level AA):** Requires that any functionality using a dragging movement for operation can also be achieved by a single pointer without dragging, unless dragging is essential. [5]
- **2.5.8 Target Size (Minimum) (Level AA):** Specifies a minimum size for pointer targets (at least 24 by 24 CSS pixels) or sufficient spacing around smaller targets to prevent accidental activation, especially on touch interfaces. [5]
- **3.2.6 Consistent Help (Level A):** Mandates that if help mechanisms (e.g., contact details, FAQs, chatbots) are repeated on multiple web pages within a set,

they occur in the same relative order to other page content. [5]

- **3.3.7 Redundant Entry (Level A):** Requires that information previously entered by or provided to the user in the same process is either auto-populated or available for the user to select, reducing the need for re-entry. [5]
- **3.3.8 Accessible Authentication (Minimum) (Level AA):** Stipulates that a cognitive function test (like remembering a password or solving a puzzle) is not required for any step in an authentication process unless an alternative method that doesn't rely on such a test is available, or a mechanism is provided to assist the user. [5]

These new criteria reflect an evolving understanding of digital accessibility, with a notable emphasis on improving usability for individuals with cognitive and learning disabilities (as seen in Redundant Entry, Accessible Authentication, and Consistent Help) and enhancing the experience on mobile and touch-based devices (addressed by Dragging Movements, Target Size, and Focus Not Obscured). This shift signals to QA teams that these specific areas warrant increased testing focus, potentially requiring the adoption of new testing techniques or a more intensive application of existing ones.

Removal of Success Criterion 4.1.1 Parsing:

A significant change in WCAG 2.2 is the obsolescence and removal of Success Criterion 4.1.1 Parsing.[6] This criterion previously required that content implemented using markup languages has complete start and end tags, elements are nested according to their specifications, elements do not contain duplicate attributes, and any IDs are unique, except where the specifications allow these features.

The removal of SC 4.1.1 acknowledges that modern browsers and assistive technologies have become more adept at handling minor parsing errors, and that strict adherence to this criterion did not always directly correlate with improved accessibility for end-users. However, this does not diminish the importance of well-formed code. The overarching principle of **Robustness**—ensuring content can be reliably interpreted by a wide variety of user agents, including assistive technologies—remains critical. QA's focus, therefore, should shift from strict syntactic validation against parsing rules to more practical testing of how assistive technologies actually interpret and interact with the content. This places an even greater emphasis on thorough testing against Success Criterion **4.1.2 Name, Role, Value**, which ensures that user interface components, their states, and properties are

programmatically determinable by assistive technologies.

1.4 The QA Professional's Role in Ensuring Digital Accessibility

The role of the Quality Assurance (QA) professional in the realm of digital accessibility is multifaceted and extends far beyond simple bug detection. [17] QA testers are pivotal in verifying that a digital product not only meets the technical requirements of WCAG 2.2 but also provides a genuinely usable and positive experience for individuals with disabilities. [17] This involves a deep understanding of accessibility principles and standards, proficiency in various testing methodologies, and a commitment to user advocacy.

Core responsibilities of QA in accessibility include:

- **Understanding Requirements:** Thoroughly comprehending the applicable WCAG 2.2 Level A and AA success criteria and how they apply to the product under test. [17]
- **Diverse Testing Approaches:** Employing a combination of automated testing to quickly identify common issues, meticulous manual testing to assess nuanced and contextual aspects, and hands-on testing with assistive technologies (AT) like screen readers, screen magnifiers, and keyboard-only navigation to simulate real-user experiences. [17] Automated tools are expected to uncover about 30-35% of potential issues, with the remainder identified through manual methods. [17]
- **Meticulous Issue Documentation:** Clearly and accurately documenting accessibility defects, including steps to reproduce, the specific WCAG SC violated, the user impact (especially for those using AT), and relevant environmental details (browser, OS, AT version). [19]
- **User Advocacy:** Championing the needs of users with disabilities throughout the development lifecycle, ensuring their perspectives are considered in design and implementation decisions. [17] This requires empathy and a user-centric mindset, focusing on the overall positive experience rather than just technical checkbox compliance.
- **Collaboration and Communication:** Effectively communicating findings to development and design teams, helping them understand the issues, and verifying fixes. [21]

The evolution of web technologies and accessibility standards means that QA professionals must engage in continuous learning. The increasing complexity of web applications, such as Single Page Applications (SPAs) and dynamic content, demands

more sophisticated manual and AT testing skills, as automated tools often struggle with these aspects.[22] Furthermore, the QA role is shifting "left," requiring earlier involvement in the development lifecycle, such as in design reviews and sprint planning, to prevent accessibility issues from being built in.[24] This proactive approach is more efficient and cost-effective than attempting to remediate deeply embedded issues late in the cycle.

Ultimately, accessibility QA is a specialized field. It demands more than general functional testing skills; it requires a nuanced understanding of how different disabilities impact user interaction with digital interfaces and practical experience in using assistive technologies to evaluate those interactions.[17] Organizations must recognize this and invest in training and resources to empower their QA teams to effectively ensure digital inclusivity.

Section 2: Comprehensive Guide to WCAG 2.2 Level A & AA Success Criteria

This section provides a detailed guide to each Web Content Accessibility Guidelines (WCAG) 2.2 Level A and Level AA success criterion. It is structured to be a practical resource for Quality Assurance (QA) professionals, offering insights into the purpose of each criterion from a testing perspective, specific methodologies for verification, common ways in which criteria are failed, and illustrative test cases. The information is organized according to the four POUR principles (Perceivable, Operable, Understandable, Robust) and their respective guidelines.

The effective testing of these criteria often relies on a combination of manual inspection, the use of browser developer tools, specialized accessibility testing tools (such as color contrast analyzers and automated checkers like Axe-core or WAVE), and direct testing with assistive technologies (AT) like screen readers (e.g., NVDA, JAWS, VoiceOver), screen magnifiers, and keyboard-only navigation. Understanding the interdependencies between criteria is also beneficial; for example, fundamental keyboard accessibility (SC 2.1.1) is a prerequisite for testing many focus-related criteria, and correct semantic information (SC 1.3.1, SC 4.1.2) is vital for meaningful screen reader testing.

WCAG 2.2 Level A & AA Success Criteria: A QA Testing Guide

SC Number & Title (Level) Full SC Text	Purpose (from a Testing Perspective)	Testing Methodologies (Manual, Tool - Assisted, Assistive Technology)	Common Failure Points	Example Test Cases (Pass/Fail)
Principle 1: Perceivable				
Guideline 1.1 Text Alternatives	Provide text alternatives for any non-text content so that it can be changed into other forms people need, such as large print, braille, speech, symbols or simpler language.			
1.1.1 Non-text Content (Level A) All non-text content that is presented to the user has a text alternative that serves the equivalent purpose, except for the situations listed below. • **Controls, Input:** If non-text content is a	To verify that all informative non-text content (images, icons, charts, etc.) has a text alternative that conveys its purpose or describes the information, ensuring users who cannot see the content (e.g., blind users, users with low vision)	**Manual:** Visually inspect all non-text elements. Use browser developer tools to check alt attributes for <img>, accessible names for icons (e.g., via aria-label or visually hidden text), and title attributes for <iframe>. **Tool-Assisted:**	Missing alt attribute on informative images. Empty alt attribute (alt="") on informative images. Non-descriptive alt text (e.g., "image," filename, "graphic"). alt text on functional images (links/buttons)	**Pass:** An image of a magnifying glass used as a search button has alt="Search". **Fail:** An informative chart is presented as an image with alt="Chart showing sales data". (Fails because it doesn't convey the *information*

control or accepts user input, then it has a name that describes its purpose. (Refer to Guideline 4.1 for additional requirements for controls and content that accepts user input.) • **Time-Based Media:** If non-text content is time-based media, then text alternatives at least provide descriptive identification of the non-text content. (Refer to Guideline 1.2 for additional requirements for media.) • **Test:** If non-text content is a test or exercise that would be invalid if presented in text, then text alternatives at least provide descriptive identification of the non-text content. • **Sensory:** If non-text content is primarily	can understand it. For controls, ensure they have an accessible name. Decorative content should be ignorable by AT. [26]	Automated checkers (Axe, WAVE) can flag missing or suspicious alt text. [27] **AT Testing:** Use a screen reader (NVDA, JAWS, VoiceOver) to listen to how non-text content is announced. Verify it's meaningful and accurate. [29]	doesn't describe the action/destination. CAPTCHA lacks a text alternative describing its purpose or a non-visual alternative. Purely decorative images have alt text instead of being marked as decorative (e.g., alt="" or CSS background image). [1]	in the chart). **Pass:** A purely decorative horizontal line image has alt="".

intended to create a specific sensory experience, then text alternatives at least provide descriptive identification of the non-text content. • **CAPTCHA:** If the purpose of non-text content is to confirm that content is being accessed by a person rather than a computer, then text alternatives that identify and describe the purpose of the non-text content are provided, and alternative forms of CAPTCHA using output modes for different types of sensory perception are provided to accommodate different disabilities. • **Decoration, Formatting, Invisible:** If non-text content is pure				

decoration , is used only for visual formatting, or is not presented to users, then it is implemented in a way that it can be ignored by assistive technology. [5]				
Guideline 1.2 Time-based Media	Provide alternatives for time-based media.			
1.2.1 Audio-only and Video-only (Prerecorded) (Level A) For prerecorded audio-only and prerecorded video-only media, the following are true, except when the audio or video is a media alternative for text and is clearly labeled as such: • **Prerecorded Audio-only:** An alternative for time-based media is provided that	To verify that users who are deaf or hard of hearing can access prerecorded audio-only content via a text transcript, and that users who are blind or have low vision can access prerecorded video-only content via a descriptive text transcript or an audio track. [32]	**Manual:** For audio-only, check for a linked or embedded text transcript. Read it to ensure it's accurate and complete (dialogue, speaker ID, significant sounds). For video-only, check for a linked/embedded transcript describing visual information or an available audio track describing the visuals. [33] **Tool-Assisted:**	Transcript missing for audio-only content. Transcript or audio description missing for video-only content. Transcript is inaccurate or incomplete (omits dialogue, speaker identification, or important sounds/visuals). Media alternative for text is not clearly labeled as such. [32]	**Pass (Audio-only):** A podcast episode page includes a link "Read full transcript" leading to an accurate text version of the audio. **Fail (Video-only):** A silent animation demonstrating a product feature has no accompanying text description or audio track explaining the visual steps.

presents equivalent information for prerecorded audio-only content. • **Prerecorded Video-only:** Either an alternative for time-based media or an audio track is provided that presents equivalent information for prerecorded video-only content. [5]		No specific tools, primarily manual review. **AT Testing:** Use a screen reader to access and read the transcript. For video-only with an audio track, listen to ensure it describes visual content.		
1.2.2 Captions (Prerecorded) (Level A) Captions are provided for all prerecorded audio content in synchronized media, except when the media is a media alternative for text and is clearly labeled as such. [5]	To verify that all prerecorded videos with audio have accurate, synchronized captions, enabling users who are deaf or hard of hearing to understand the audio content, including dialogue, speaker identification, and important non-speech sounds. [35]	**Manual:** Play the video. Turn on captions. Verify captions are present, accurate (match spoken words and identify speakers), synchronized with the audio, and include important non-speech sounds (e.g., "[music]", "[laughter]"). [36] **Tool-Assisted:** Some video platforms provide caption editing tools. Automated	Captions are missing. Captions are inaccurate (do not match dialogue, miss important sounds). Captions are not synchronized with the audio. Speaker identification is missing when not obvious. Captions omit or misrepresent important non-speech sounds.	**Pass:** A prerecorded webinar video has closed captions that accurately transcribe all speech, identify different speakers, and note "[audience applause]". **Fail:** A product demonstration video has auto-generated captions that contain numerous errors in transcribing technical terms and do not

		caption generation services exist but their output *must* be manually reviewed and corrected for accuracy. **AT Testing:** Not directly for caption content, but ensure caption controls are accessible.	[35]	indicate who is speaking.
1.2.3 Audio Description or Media Alternative (Prerecorded) (Level A) An alternative for time-based media or audio description of the prerecorded video content is provided for synchronized media, except when the media is a media alternative for text and is clearly labeled as such. [5]	To verify that users who are blind or have low vision can access the visual information in prerecorded synchronized media, either through an audio description track narrating visual events or a full text alternative (like a screenplay) that includes descriptions of visual content. [38]	**Manual:** For videos, check if an audio description track is available and can be selected. Listen to it to ensure it describes important visual information not conveyed in the main audio. Alternatively, check for a link to a text-based media alternative and verify it accurately describes both audio and visual content in sequence. [39] **Tool-Assisted:** Media players should	No audio description or text alternative is provided for video content with important visuals. Audio description is provided but is inaccurate or omits key visual information. Text alternative is provided but does not adequately describe visual content or is not synchronized with the media's timeline. Media alternative for text is not clearly labeled	**Pass:** A movie scene shows a character silently reacting to an event; the audio description track says, "Sarah gasps, her eyes wide with surprise." **Fail:** A training video demonstrates a complex physical task with no audio description track and no text alternative describing the visual steps performed.

		support selection of audio description tracks. **AT Testing:** Use a screen reader to access the text-based media alternative if provided.	as such. [38]	
1.2.4 Captions (Live) (Level AA) Captions are provided for all live audio content in synchronized media. [5]	To verify that all live synchronized media (e.g., webcasts, live presentations) have real-time captions, enabling users who are deaf or hard of hearing to participate and understand the audio content as it happens. [41]	**Manual:** During a live event, enable captions. Verify their presence, general accuracy (allowing for real-time transcription errors but capturing the essence), and synchronization with the live audio. Check for speaker identification and inclusion of important non-speech sounds if feasible in a live context. [41] **Tool-Assisted:** Relies on live captioning services (e.g., CART reporters, AI-based live captioning	Live captions are not provided for a live webcast. Live captions are significantly delayed or highly inaccurate, making them unusable. Speaker identification is consistently missing in a multi-speaker live event. [41]	**Pass:** A live-streamed conference presentation includes real-time captions that accurately reflect the speaker's words and identify the speaker when they change. **Fail:** A live Q&A session is broadcast without any form of live captioning.

		platforms). QA checks the output and integration. **AT Testing:** Ensure caption controls are accessible.		
1.2.5 Audio Description (Prerecorded) (Level AA) Audio description is provided for all prerecorded video content in synchronized media. [5]	To verify that all prerecorded synchronized media includes audio description for its visual information, enabling users who are blind or have low vision to understand the visual context, actions, characters, scene changes, and on-screen text not conveyed in the main soundtrack. [42]	**Manual:** Play the video and select the audio description track. Listen to ensure it accurately and adequately describes all significant visual information that isn't part of the main audio. Verify descriptions fit into natural pauses in dialogue. [42] **Tool-Assisted:** Media players must support selection of audio description tracks. **AT Testing:** Not directly for consuming audio description, but ensure controls to enable it are accessible.	Audio description track is missing for a video with significant visual content. Audio description is present but omits important visual details or is inaccurate. Audio description overlaps with dialogue or important sounds, making both hard to understand. [42]	**Pass:** A documentary video includes an audio description track that describes the scenery, actions of people shown, and any text overlays that appear on screen. **Fail:** An instructional video showing how to assemble a product lacks audio description for the visual steps being demonstrated.

Guideline 1.3 Adaptable	Create content that can be presented in different ways (for example simpler layout) without losing information or structure.			
1.3.1 Info and Relationships (Level A) Information, structure , and relationships conveyed through presentation can be programmatically determined or are available in text. [5]	To verify that the semantic structure (e.g., headings, lists, tables, form labels and fieldsets) and relationships conveyed visually are also programmatically available to assistive technologies, ensuring users can understand and navigate content even if they cannot perceive the default visual presentation. [43]	**Manual:** Use browser developer tools to inspect HTML structure. Check for correct use of heading tags (<h1>-<h6>), list elements (<ul>, <ol>, <dl>), table markup (<table>, <th>, <td>, <caption>, scope), and form labeling (<label>, <fieldset>, <legend>). Verify that visual relationships (e.g., grouping of related items) are reflected in the code. [44] **Tool-Assisted:** Automated checkers (Axe, WAVE) can identify some structural and ARIA misuse	Headings used for visual styling only, not for actual document structure (e.g., using bold text instead of <h2>). Lists created with plain text and hyphens/asterisks instead of <ul> or <ol>. Data tables lacking <th> for headers, or scope attributes, or <caption>. Form inputs without programmatically associated <label> elements. Related form controls not grouped with <fieldset> and <legend>. Relying on visual proximity alone	**Pass:** A data table uses <th> elements with scope attributes for row and column headers, and a <caption> element to describe the table. **Fail:** A form uses bolded text next to input fields as visual labels, but there are no <label> elements programmatically linking them to the inputs.

		issues. **AT Testing:** Use a screen reader to navigate by headings, lists, tables, and forms. Verify that structure is announced correctly and relationships (e.g., form label to input, table header to cell) are clear.	to imply relationships. [43]	
1.3.2 Meaningful Sequence (Level A) When the sequence in which content is presented affects its meaning, a correct reading sequence can be programmatically determined. [5]	To verify that when the order of content presentation is essential for understanding, that order is preserved when accessed by assistive technologies or when CSS is disabled, ensuring the logical flow is maintained. [46]	**Manual:** Disable CSS (e.g., using browser developer tools or extensions). Check if the content reads in a logical order. View the source order in the DOM inspector. **Tool-Assisted:** Some tools can linearize the page or show the DOM order. **AT Testing:** Use a screen reader to read through the page content. Verify that the reading order matches the logical flow of information	Content visually arranged with CSS (e.g., multi-column layouts, sidebars) has a DOM order that does not make sense when read linearly. Tables used for layout result in a confusing reading order when linearized. Important information is out of sequence, leading to misunderstanding. [46]	**Pass:** A two-column news article, when CSS is disabled, reads the first column from top to bottom, then the second column from top to bottom. **Fail:** A form is laid out visually using a table, but the tab order and screen reader reading order jump illogically between fields.

		as intended by the visual design.		
1.3.3 Sensory Characteristics (Level A) Instructions provided for understanding and operating content do not rely solely on sensory characteristics of components such as shape, color, size, visual location, orientation, or sound. (Note: For requirements related to color, refer to Guideline 1.4.)[5]	To verify that instructions for operating content do not depend exclusively on sensory information (e.g., "click the round button," "item on the right"), ensuring users who cannot perceive these characteristics (e.g., blind users, users with low vision, colorblind users) can still understand and act on the instructions. [47]	**Manual:** Review all instructional text. Identify any instructions that refer to shape, color, size, visual location, orientation, or sound. For each, check if a non-sensory alternative is also provided (e.g., a visible text label, a programmatic name). [48] **Tool-Assisted:** Not directly applicable for identifying reliance on sensory characteristics, but tools can help inspect labels and names. **AT Testing:** Use a screen reader. If instructions say "click the red button," verify the button has an accessible name like "Submit button" that the screen	Instructions refer to "the button on the left" without providing a text label for the button. Instructions say "Press the square icon" when the icon has no text label or accessible name. Error messages are indicated only by a red border around a field, with instructions like "correct the fields highlighted in red." [47]	**Pass:** Instructions state: "To submit the form, click the 'Submit' button located at the bottom right." (Provides both location and label). **Fail:** Instructions state: "Click the large green circle to continue." and the circle has no text label or accessible name.

		reader announces, allowing identification without seeing the color.		
1.3.4 Orientation (Level AA) Content does not restrict its view and operation to a single display orientation, such as portrait or landscape, unless a specific display orientation is essential. [5]	To verify that content can be viewed and operated in both portrait and landscape orientations, unless a specific orientation is essential for the content's use (e.g., a banking app for check deposit, a piano app). This benefits users who have devices mounted in a fixed orientation or who prefer a specific orientation for readability. [50]	**Manual:** On a device that supports orientation changes (e.g., smartphone, tablet), load the content. Rotate the device between portrait and landscape. Verify that all content and functionality remains available and usable in both orientations without loss of information or requiring excessive scrolling. **Tool-Assisted:** Browser developer tools can simulate different device orientations. **AT Testing:** Test with AT in both orientations to ensure continued	Content forces a specific orientation (e.g., locks to portrait mode) when it's not essential. Content becomes unusable or information is lost when orientation changes (e.g., buttons disappear, text is cut off). [50]	**Pass:** A news article website displays correctly and is fully navigable in both portrait and landscape modes on a tablet. **Fail:** A messaging app forces landscape mode on a phone, even though messaging could easily work in portrait.

		operability.		
1.3.5 Identify Input Purpose (Level AA) The purpose of each input field collecting information about the user can be programmatically determined when: • The input field serves a purpose identified in the Input Purposes for User Interface Components section; and • The content is implemented using technologies with support for identifying the expected meaning for form input data. [5]	To verify that the purpose of input fields collecting common user information (name, address, email, etc.) is programmatically defined using appropriate autocomplete attributes. This allows browsers and assistive technologies to autofill fields and present familiar icons, aiding users with cognitive and motor impairments. [51]	**Manual:** Inspect the HTML of forms collecting user data. Check for the presence and correctness of autocomplete attributes on input fields, referencing the list of valid tokens in WCAG's Input Purposes section. [48] **Tool-Assisted:** Browser developer tools can show computed autocomplete values. Some accessibility checkers might flag missing or invalid autocomplete attributes. **AT Testing:** Test with browsers/AT that support autofill and icon display based on autocomplete. Verify that fields are correctly identified and autofilled where	autocomplete attribute is missing for common input fields like "name" or "email." Incorrect or non-standard autocomplete tokens are used. autocomplete is used on fields not collecting information about the user. [51]	**Pass:** An input field for the user's first name has autocomplete="given-name". **Fail:** An input field for "Credit Card Number" has autocomplete="cc-name" (valid is cc-number) or autocomplete="creditcard".

		appropriate.		
Guideline 1.4 Distinguishable	Make it easier for users to see and hear content including separating foreground from background.			
1.4.1 Use of Color (Level A) Color is not used as the only visual means of conveying information, indicating an action, prompting a response, or distinguishing a visual element. (Note: This success criterion addresses color perception specifically. Other forms of perception are covered in Guideline 1.3 including programmatic access to color and other visual presentation coding.) [5]	To verify that information conveyed with color is also available through other visual means (e.g., text, icons, patterns, underlining), ensuring users who are colorblind or have low vision can perceive it. [52]	**Manual:** Identify all instances where color is used to convey information (e.g., red for errors, green for success, link color). Check if another visual cue (text label, icon, underline, pattern) is present. Use a color blindness simulator tool or view the page in grayscale to confirm the alternative cue is effective. [53] **Tool-Assisted:** Color blindness simulators (browser extensions or standalone). **AT Testing:** Not directly for color	Required form fields are indicated only by red text. Links in text are distinguishable from surrounding text only by color (no underline or other visual cue). Chart data is differentiated only by the color of slices/bars, without patterns or labels. [52]	**Pass:** Required form fields are marked with red text *and* an asterisk (*). **Fail:** A graph uses different colored lines to represent different data sets, but provides no legend with patterns or textual labels to distinguish the lines without relying on color.

		perception, but screen readers should announce the alternative textual cues.		
1.4.2 Audio Control (Level A) If any audio on a web page plays automatically for more than 3 seconds, either a mechanism is available to pause or stop the audio, or a mechanism is available to control audio volume independently from the overall system volume level. (Note: Since any content that does not meet this success criterion can interfere with a user's ability to use the whole page, all content on the web page (whether or not it is used to meet other success criteria) must meet this	To verify that users can stop or pause any audio that plays automatically for more than 3 seconds, or control its volume independently, preventing interference with screen readers or unwanted distractions. [55]	**Manual:** Load pages and listen for any audio that starts automatically. If it plays for >3 seconds, look for a visible, keyboard-accessible control to pause, stop, or adjust its volume independently of system volume. Test the control. [49] **Tool-Assisted:** Browser developer tools can help identify <audio> or <video> elements with an autoplay attribute. **AT Testing:** Use a screen reader. If audio autoplays, check if it interferes with the screen reader and if the audio	Background music autoplays for >3 seconds with no on-page control to stop or mute it. An autoplaying video with sound has no visible pause/stop or volume controls, or these controls only affect system volume. [55]	**Pass:** A website has background music that autoplays but provides a clearly labeled "Mute Music" button at the top of the page. **Fail:** An embedded video advertisement starts playing audio automatically for 10 seconds with no user controls visible on the player.

success criterion. See Conformance Requirement 5: Non-Interference.) [5]		control is easily found and operated.		
1.4.3 Contrast (Minimum) (Level AA) The visual presentation of text and images of text has a contrast ratio of at least 4.5:1, except for the following: • **Large Text:** Large-scale text and images of large-scale text have a contrast ratio of at least 3:1; • **Incidental:** Text or images of text that are part of an inactive user interface component, that are pure decoration, that are not visible to anyone, or that are part of a picture that contains significant other visual content, have no contrast	To verify that text and images of text have sufficient luminance contrast with their background, making them readable for users with moderately low vision or color vision deficiencies. [56]	**Manual:** Identify all text and images of text. Determine if they are regular or large-scale. Exclude incidental text and logotypes. **Tool-Assisted:** Use a color contrast analyzer tool (e.g., TPGi CCA, WebAIM Contrast Checker, browser dev tools) to measure the ratio between foreground and background colors. For regular text, ratio must be ≥4.5:1. For large text (18pt/24px or 14pt/18.5px bold), ratio must be ≥3:1. [57] **AT Testing:** Screen magnifiers may be used by users who need	Regular text has a contrast ratio below 4.5:1 (e.g., light grey text on a white background). Large text has a contrast ratio below 3:1. Text on background images has insufficient contrast in some areas. Placeholder text in input fields has insufficient contrast. [56]	**Pass:** Body text (16px) is dark grey (#333333) on a white (#FFFFFF) background (Ratio >4.5:1). A large heading (24px bold) is medium grey (#767676) on white (Ratio > 3:1). **Fail:** Link text (16px) is light blue (#88C9F9) on a white background (Ratio <4.5:1).

requirement. • **Logotypes:** Text that is part of a logo or brand name has no contrast requirement. [5]		higher contrast; ensure the base contrast is sufficient.		
1.4.4 Resize text (Level AA) Except for captions and images of text, text can be resized without assistive technology up to 200 percent without loss of content or functionality. [5]	To verify that users can increase text size up to 200% using browser zoom or text resize features without loss of content or functionality, benefiting users with low vision. [59]	**Manual:** Use the browser's zoom feature to increase page zoom to 200%. Check that all text scales, content does not overlap or get truncated, and all functionality remains available. Alternatively, if browser supports text-only resize, test that. [49] **Tool-Assisted:** Browser developer tools can simulate different zoom levels and inspect layout issues. **AT Testing:** Users with low vision might use system-level text size settings or screen	Text is truncated or overlaps other content when zoomed to 200%. Horizontal scrollbars appear for text content when zoomed to 200% (see also SC 1.4.10 Reflow). Functionality (e.g., buttons, links) becomes obscured or unusable at 200% zoom. Text is implemented in a way that prevents browser resizing (e.g., fixed-size containers for text in pixels). [60]	**Pass:** All text on the page scales to 200%, content reflows, and all buttons and links are still visible and usable. **Fail:** When zoomed to 200%, text in a navigation menu overlaps, making some links unreadable and unclickable.

		magnifiers; ensure browser zoom doesn't break content before these are even applied.		
1.4.5 Images of Text (Level AA) If the technologies being used can achieve the visual presentation, text is used to convey information rather than images of text except for the following: • **Customizable:** The image of text can be visually customized to the user's requirements; • **Essential:** A particular presentation of text is essential to the information being conveyed. (Note: Logotypes (text that is part of a logo or brand name) are	To verify that actual text is used instead of images of text whenever possible, allowing users (especially those with low vision or cognitive disabilities) to customize text presentation (size, color, font, spacing). Images of text are only acceptable if customizable or essential (e.g., logos). [61]	**Manual:** Identify all instances where text is presented as an image. Determine if the same visual presentation could be achieved with CSS and actual text. If so, it's likely a failure unless it meets an exception. Check if "essential" images of text (e.g., logos) have appropriate alt text (SC 1.1.1). **Tool-Assisted:** Browser developer tools can help distinguish actual text from images of text. **AT Testing:** Screen readers will announce the alt text for	Using an image for a heading just to achieve a specific font style that could be achieved with web fonts and CSS. Using images for all navigation menu items when text styled with CSS would suffice. An informative diagram uses images of text for labels instead of actual text overlays. [61]	**Pass:** A company logo containing stylized text is an image. This is 'essential'. **Fail:** A website uses an image to display its main navigation links like "Home", "About Us", "Services" solely for a custom font effect.

considered essential.) [5]		images of text. Verify this is equivalent. Test if text in images scales poorly or pixelates with zoom, compared to actual text.		
1.4.10 Reflow (Level AA) Content can be presented without loss of information or functionality, and without requiring scrolling in two dimensions for: • Vertical scrolling content at a width equivalent to 320(https://www.w3.org/TR/WCAG22/#dfn-css-pixels); • Horizontal scrolling content at a height equivalent to 256(https://www.w3.org/TR/WCAG22/#dfn-css-pixels). Except for parts of the content which require two-	To verify that content reflows to a single column when zoomed to 400% (or viewport width of 320 CSS pixels for vertical content / 256 CSS pixels height for horizontal content), eliminating the need for two-dimensional scrolling, unless 2D layout is essential. This benefits users with low vision who magnify content. [62]	**Manual:** Set browser viewport to 1280px wide and zoom to 400% (or set viewport directly to 320px wide). Check for horizontal scrollbars for vertically scrolling content. Check for loss of content or functionality. For horizontal content, use 1024px height and zoom to 400% (or 256px height) and check for vertical scrollbars. [49] **Tool-Assisted:** Browser developer tools for setting viewport size	Text content requires horizontal scrolling at 320 CSS pixels width. Fixed-width containers or elements cause content to be cut off or require horizontal scrolling. Sticky headers/footers take up too much vertical space at 400% zoom, obscuring content. Multi-column layouts do not convert to a single column. [62]	**Pass:** At 320 CSS pixels width, a webpage's main content and sidebar stack vertically, and all text wraps, requiring only vertical scrolling. **Fail:** At 320 CSS pixels width, a line of text in a paragraph extends beyond the viewport, introducing a horizontal scrollbar for the entire page.

dimensional layout for usage or meaning. (Note 1: 320 CSS pixels is equivalent to a starting viewport width of 1280 CSS pixels wide at 400% zoom. For web content which is designed to scroll horizontally (e.g., with vertical text), 256 CSS pixels is equivalent to a starting viewport height of 1024 CSS pixels at 400% zoom. Note 2: Examples of content which requires two-dimensional layout are images required for understanding (such as maps and diagrams), video, games, presentations, data tables (not individual cells), and interfaces where it is necessary to keep toolbars in view while		and zoom. **AT Testing:** Test with screen magnifiers to understand the user experience when content doesn't reflow.		

manipulating content. It is acceptable to provide two-dimensional scrolling for such parts of the content.) [5]				
1.4.11 Non-text Contrast (Level AA) The visual presentation of the following have a contrast ratio of at least 3:1 against adjacent color(s): • **User Interface Components:** Visual information required to identify user interface components and states, except for inactive components or where the appearance of the component is determined by the user agent and not modified by the author; • **Graphical Objects:** Parts of graphics	To verify that active user interface components (like buttons, input borders) and meaningful graphical objects (like icons, chart elements) have a contrast ratio of at least 3:1 against their adjacent colors, making them distinguishable for users with low vision or color deficiencies. [63]	**Manual:** Identify all active UI components and meaningful graphical objects. **Tool-Assisted:** Use a color contrast analyzer tool to measure the contrast ratio of the component's visual boundary or identifying parts against adjacent background(s). For graphical objects, measure parts required for understanding against their background. [57] **AT Testing:** Not directly for contrast, but ensures components are perceivable enough to be	Borders of input fields have less than 3:1 contrast with the adjacent page background. An icon used as a button (without text) has less than 3:1 contrast with its background. The visual indicator of a selected state (e.g., a checkmark in a checkbox) has less than 3:1 contrast with the checkbox's internal background. Important lines or segments in a chart/diagram have less than 3:1 contrast with their background. [63]	**Pass:** A button has a visible border with a 3.5:1 contrast ratio against the page background. A selected radio button shows a dark inner circle that has 4:1 contrast with the radio button's white interior. **Fail:** A search icon button is light grey on a slightly darker grey background, with a contrast ratio of 2:1.

required to understand the content, except when a particular presentation of graphics is essential to the information being conveyed. [5]		interacted with by users who might rely on some visual cues.		
1.4.12 Text Spacing (Level AA) In content implemented using markup languages that support the following text style properties, no loss of content or functionality occurs by setting all of the following and by changing no other style property: • Line height (line spacing) to at least 1.5 times the font size; • Spacing following paragraphs to at least 2 times the font size; • Letter spacing (tracking) to at	To verify that users can override author-defined text spacing (line height, paragraph spacing, letter spacing, word spacing) to specific minimums without content being cut off, overlapping, or functionality being lost. This benefits users with low vision or cognitive disabilities like dyslexia who require increased spacing for readability. [64]	**Manual:** Use a browser extension (e.g., "Text Spacing" bookmarklet by Steve Faulkner) or developer tools to apply the specified spacing metrics to the page content. **Tool-Assisted:** Text spacing bookmarklets or custom CSS applied via developer tools. [49] **AT Testing:** Not directly for applying spacing, but to check if functionality breaks when spacing is applied by the user.	Text gets truncated or cut off within its container when spacing is applied. Text overlaps other text or UI elements. Interactive elements become unusable or hidden. Content requires horizontal scrolling after spacing adjustments (violating 1.4.10 Reflow as well). [64]	**Pass:** After applying the WCAG text spacing metrics, all text remains readable, within its containers, does not overlap, and all interactive elements are still functional. **Fail:** Applying the text spacing metrics causes words in a button to be cut off, making the button's label unreadable.

least 0.12 times the font size; • Word spacing to at least 0.16 times the font size. Exception: Human languages and scripts that do not make use of one or more of these text style properties in written text can conform using only the properties that exist for that combination of language and script. [5]				
1.4.13 Content on Hover or Focus (Level AA) Where receiving and then removing pointer hover or keyboard focus triggers additional content to become visible and then hidden, the following are true: • **Dismissible:** A mechanism is available to dismiss the	To verify that additional content appearing on hover or focus (e.g., tooltips, sub-menus) is dismissible without moving focus, hoverable if triggered by hover, and persistent until dismissed or no longer valid. This helps users with low vision (magnificati on users) and	**Manual:** *Dismissible:* Trigger content with hover/focus. Try to dismiss it using the Escape key without moving the pointer/focus. Check if it obscures other content. *Hoverable:* If triggered by hover, move the pointer onto the new content. Verify it doesn't	Tooltip disappears when trying to move the mouse over it. Pop-up menu triggered on focus cannot be dismissed with the Escape key and traps focus. Additional information panel disappears too quickly before it can be read. [59]	**Pass:** A custom tooltip appears on hover, remains visible when the mouse moves onto it, and can be dismissed by pressing the Escape key. **Fail:** A sub-menu appears on hover over a navigation item but disappears if the user tries to move the mouse pointer down into the

additional content without moving pointer hover or keyboard focus, unless the additional content communicates an input error or does not obscure or replace other content; • **Hoverable:** If pointer hover can trigger the additional content, then the pointer can be moved over the additional content without the additional content disappearing; • **Persistent:** The additional content remains visible until the hover or focus trigger is removed, the user dismisses it, or its information is no longer valid. **Exceptions:** The visual presentation of the additional content is	motor impairments. [59]	disappear. *Persistent:* Verify content stays visible until hover/focus is removed, user dismisses it, or info is invalid. [59] **Tool-Assisted:** Browser developer tools to inspect event handlers and CSS. **AT Testing:** Test with keyboard and screen magnifiers to assess usability.		sub-menu.

controlled by the user agent and is not modified by the author. [5]				
Principle 2: Operable				
Guideline 2.1 Keyboard Accessible	Make all functionality available from a keyboard.			
2.1.1 Keyboard (Level A) All functionality of the content is operable through a keyboard interface without requiring specific timings for individual keystrokes, except where the underlying function requires input that depends on the path of the user's movement and not just the endpoints. (Note 1: This exception relates to the underlying	To verify that all interactive elements and functionalities can be operated using only a keyboard, without needing specific timing for keystrokes. This is crucial for users who cannot use a mouse (e.g., users with motor impairments, blind users relying on screen readers). [66]	**Manual:** Disconnect or ignore the mouse. Use Tab, Shift+Tab, Enter, Spacebar, and Arrow keys to navigate to and operate all links, buttons, form controls, custom widgets (sliders, carousels, menus), and media player controls. Ensure all actions can be performed. [67] **Tool-Assisted:** Browser developer tools can highlight focusable elements. Some automated tools	Interactive elements are not focusable via keyboard (cannot be tabbed to). Focusable elements cannot be activated using Enter or Spacebar. Custom widgets (e.g., sliders, date pickers) lack full keyboard support for all their functions. Drag-and-drop functionality has no keyboard alternative. [66]	**Pass:** All links, buttons, and form fields are reachable and operable using Tab, Enter, and Spacebar. A custom slider can be controlled using arrow keys. **Fail:** A "Close" button on a modal dialog can only be activated by a mouse click and not with Enter or Spacebar when focused.

function, not the input technique. For example, if using handwriting to enter text, the input technique (handwriting) requires path-dependent input but the underlying function (text input) does not. Note 2: This does not forbid and should not discourage providing mouse input or other input methods in addition to keyboard operation.) [65]		can check for onclick without equivalent keyboard handlers. **AT Testing:** Screen readers rely heavily on keyboard interaction; testing with them can reveal keyboard accessibility issues.		
2.1.2 No Keyboard Trap (Level A) If keyboard focus can be moved to a component of the page using a keyboard interface, then focus can be moved away from that component using only a keyboard interface, and, if it requires more	To verify that keyboard focus does not get "trapped" within any component (e.g., embedded plugin, modal dialog, widget), ensuring keyboard-only users can always navigate away from it. If non-standard keys are needed to exit, users must be	**Manual:** Use the Tab key to navigate into various components (modals, embedded media players, complex widgets). Attempt to Tab out of the component to the next/previous element on the page. If standard	Focus gets stuck within a modal dialog, and neither Tab, Shift+Tab, nor Escape key moves focus outside. An embedded video player traps keyboard focus, preventing navigation to the rest of the page. A custom widget requires a	**Pass:** A modal dialog traps focus within itself (good for modals), but the "Close" button or Escape key successfully closes the dialog and returns focus to a logical place on the page. **Fail:** After tabbing into an embedded chat widget, the Tab key only cycles

than unmodified arrow or tab keys or other standard exit methods, the user is advised of the method for moving focus away. (Note: Since any content that does not meet this success criterion can interfere with a user's ability to use the whole page, all content on the web page (whether it is used to meet other success criteria or not) must meet this success criterion. See Conformance Requirement 5: Non-Interference.) [5]	informed. [69]	Tab/Shift+Tab doesn't work, check for instructions on how to exit (e.g., using Escape key). [69] **Tool-Assisted:** Some tools might highlight focus traps in simple scenarios. **AT Testing:** Screen reader users are particularly affected by keyboard traps. Test navigation into and out of components with a screen reader active.	specific, un-documented key combination to exit. [69]	within the widget, and there's no documented way to return focus to the main page.
2.1.4 Character Key Shortcuts (Level A) If a keyboard shortcut is implemented in content using only letter (including upper- and lower-case	To verify that single-character key shortcuts (e.g., 'm' for mute) can be turned off, remapped to include a modifier key (Ctrl, Alt, etc.), or are only	**Manual:** Identify any single-character key shortcuts. Test if they activate globally. If so, look for settings to turn them off or remap them (e.g., to Ctrl + character). If no	A webmail application uses 'r' to reply and 'a' to reply-all globally, and these cannot be turned off or remapped, causing speech input users to accidentally	**Pass:** A web application uses single-letter shortcuts for navigating a list, but these shortcuts are only active when the list itself has keyboard focus. Or, the

letters), punctuation, number, or symbol characters, then at least one of the following is true: • **Turn off:** A mechanism is available to turn the shortcut off; • **Remap:** A mechanism is available to remap the shortcut to include one or more non-printable keyboard keys (e.g., Ctrl, Alt); • **Active only on focus:** The keyboard shortcut for a user interface component is only active when that component has focus. [5]	active when the specific component they control has focus. This prevents accidental activation by speech input users or users prone to accidental key presses. [70]	such settings, check if the shortcut only works when a specific UI component has focus. **Tool-Assisted:** No specific tools, requires understanding of application functionality. **AT Testing:** Use speech input software. Dictate text containing characters used as shortcuts and observe if unintended actions are triggered.	trigger actions. A media player uses spacebar to play/pause globally, interfering with typing spaces in a nearby chat input. [70]	application provides a settings panel to disable all single-key shortcuts. **Fail:** A website uses the 's' key to jump to the search bar from anywhere on the page, with no way to disable or remap this shortcut.
Guideline 2.2 Enough Time	Provide users enough time to read and use content.			
2.2.1 Timing Adjustable (Level A) For each time limit that is set	To verify that for any content-imposed time limits (e.g., session	**Manual:** Identify any time limits (e.g., session timeout warnings, timed	A banking session times out after 5 minutes of inactivity with	**Pass:** An online form warns the user 60 seconds before a session timeout and

by the content, at least one of the following is true: • **Turn off:** The user is allowed to turn off the time limit before encountering it; or • **Adjust:** The user is allowed to adjust the time limit before encountering it over a wide range that is at least ten times the length of the default setting; or • **Extend:** The user is warned before time expires and given at least 20 seconds to extend the time limit with a simple action (for example, "press the space bar"), and the user is allowed to extend the time limit at least ten times; or • **Real-time Exception:** The time limit is a required part of a real-time event (for	timeouts, timed quizzes), users can turn off, adjust, or extend the limit, unless the time limit is essential (e.g., auction) or very long (>20 hours). This benefits users who need more time to read, comprehend, or interact. [71]	form submissions). Check if options exist to turn off the limit, adjust it (by at least 10x default), or extend it (with at least 20s warning and 10+ extensions). Verify exceptions (real-time, essential, >20hrs) are correctly applied. **Tool-Assisted:** Not directly applicable for testing adjustability, but tools might help identify scripts setting timeouts. **AT Testing:** Ensure warnings and extension mechanisms are accessible to AT users.	no warning or option to extend. An online test has a fixed 30-minute limit with no option to request more time. Content updates (e.g., news feed) refresh too quickly with no user control to slow down or pause. [71]	provides a button to extend the session by another 15 minutes, repeatable multiple times. **Fail:** A news website automatically refreshes its homepage content every 30 seconds with no mechanism for the user to pause or adjust this timing.

example, an auction), and no alternative to the time limit is possible; or • **Essential Exception:** The time limit is essential and extending it would invalidate the activity; or • **20 Hour Exception:** The time limit is longer than 20 hours. [5]				
2.2.2 Pause, Stop, Hide (Level A) For moving, blinking, scrolling, or auto-updating information, all of the following are true: • **Moving, blinking, scrolling:** For any moving, blinking or scrolling information that (1) starts automatically, (2) lasts more than five seconds, and (3) is presented in parallel with other content,	To verify that users can pause, stop, or hide any moving, blinking, scrolling content that starts automatically, lasts >5s, and is parallel to other content. For auto-updating content, users must be able to pause, stop, hide, or control update frequency. This prevents distractions, especially for users with attention or cognitive	**Manual:** Identify any auto-starting moving, blinking, scrolling, or auto-updating content. If it lasts >5s (for moving/blinking/scrolling) and is parallel to other content, look for pause/stop/hide controls. For auto-updating content, also check for frequency controls. Test these controls. Verify "essential" exceptions. [65] **Tool-Assisted:**	An animated banner ad plays for >5s with no pause/stop button. A news ticker scrolls continuously with no user control. A stock price feed auto-updates every few seconds with no way to pause or adjust frequency. [65]	**Pass:** A carousel of images autoplays but has clear "Pause" and manual navigation buttons. An auto-advancing slideshow stops after 5 seconds or provides a "Pause" button. **Fail:** A website background is an auto-playing video loop lasting more than 5 seconds with no mechanism to pause or stop it.

there is a mechanism for the user to pause, stop, or hide it unless the movement, blinking, or scrolling is part of an activity where it is essential; and • **Auto-updating:** For any auto-updating information that (1) starts automatically and (2) is presented in parallel with other content, there is a mechanism for the user to pause, stop, or hide it or to control the frequency of the update unless the auto-updating is part of an activity where it is essential. [5]	disabilities. [65]	Browser developer tools can help identify animations or scripts causing movement. **AT Testing:** Check if moving content interferes with screen reader usability. Ensure controls are AT-accessible.		
Guideline 2.3 Seizures and Physical Reactions	Do not design content in a way that is known to cause seizures or physical reactions.			

2.3.1 Three Flashes or Below Threshold (Level A) Web pages do not contain anything that flashes more than three times in any one second period, or the flash is below the general flash and red flash thresholds. (Note: Since any content that does not meet this success criterion can interfere with a user's ability to use the whole page, all content on the web page (whether or not it is used to meet other success criteria) must meet this success criterion. See Conformance Requirement 5: Non-Interference.) [5]	To verify that no web page content flashes more than three times per second, or if it does, the flash is below defined general and red flash thresholds (in terms of size and luminance change). This prevents triggering seizures in individuals with photosensitive epilepsy. [72]	**Manual:** Visually inspect for any flashing content (videos, animations, GIFs). If flashing is observed, try to count flashes per second. **Tool-Assisted:** Use a tool like the Photosensitive Epilepsy Analysis Tool (PEAT) to analyze video or animated content for flash frequency and intensity against WCAG thresholds. [74] **AT Testing:** Not directly applicable for seizure testing, but ensure alternatives are available if flashing content cannot be avoided and must be hidden.	A banner advertisement contains rapid strobing effects flashing more than 3 times per second. A video game animation includes large, bright explosions that flash repeatedly at a high frequency. Content that blinks faster than 3 times per second and is large enough. [73]	**Pass:** A video contains a lightning effect that flashes only twice in one second. An animation has a small flashing indicator that is below the general flash threshold for size. **Fail:** An animated GIF on a homepage flashes rapidly (e.g., 5 times per second) and occupies a significant portion of the screen.
Guideline 2.4 Navigable	Provide ways to help users			

	navigate, find content, and determine where they are.			
2.4.1 Bypass Blocks (Level A) A mechanism is available to bypass blocks of content that are repeated on multiple web pages. [5]	To verify that a mechanism (e.g., "skip to main content" link, use of headings or ARIA landmarks) exists to allow keyboard and screen reader users to bypass blocks of content repeated on multiple pages (like headers, navigation menus), enabling quicker access to the main content. [76]	**Manual:** Using keyboard only (Tab key), check if the first focusable item is a "skip" link. Activate it and verify it moves focus to the main content. Alternatively, check for semantic structure (headings, ARIA landmarks) that AT can use to bypass blocks. [76] **Tool-Assisted:** Browser extensions can show headings or landmarks. **AT Testing:** Use a screen reader to navigate by headings or landmarks to bypass repeated blocks. Verify "skip" links work as expected with the screen reader.	No "skip to main content" link is provided. Main content area is not identifiable by a heading or landmark. A "skip" link is present but doesn't work correctly or doesn't move focus appropriately. [76]	**Pass:** The first focusable element on the page is a "Skip to Main Content" link that, when activated, moves focus to the start of the primary page content. **Fail:** A website has a large navigation menu repeated on every page, but no mechanism for keyboard users to skip directly to the main content.

2.4.2 Page Titled (Level A) Web pages have titles that describe topic or purpose. [5]	To verify that each web page has a unique and descriptive <title> element that accurately reflects the page's topic or purpose, helping users (especially AT users and those with multiple tabs open) to orient themselves and identify content. [77]	**Manual:** Check the browser tab or window title bar for the page title. Ensure it is descriptive and unique for different pages. For SPAs, verify the title updates dynamically as the main content/view changes. [78] **Tool-Assisted:** Automated checkers can flag missing or duplicate page titles. Browser developer tools show the <title> element. **AT Testing:** Use a screen reader. Listen for the page title announcement upon page load or view change. Verify it's meaningful.	Page <title> is missing or empty. Page title is generic (e.g., "Untitled Page," "Homepage") for multiple distinct pages. Page title does not accurately describe the page's content or purpose. In an SPA, the title doesn't update when the view changes significantly. [77]	**Pass:** A product page titled "Product Name - Category - Site Name." An SPA view for "User Profile" updates the browser title to "User Profile - Site Name." **Fail:** All pages on a website have the same title: "Welcome to Our Website."
2.4.3 Focus Order (Level A) If a web page can be navigated sequentially and the navigation sequences affect meaning	To verify that when navigating with a keyboard, focus moves through interactive elements in an order that is logical,	**Manual:** Use the Tab key (and Shift+Tab) to navigate through all interactive elements on the page. Observe the path focus	Focus jumps erratically around the page. Focus order does not match the visual reading order, causing	**Pass:** In a form, tabbing moves sequentially from the first input field to the last, then to the submit button. When a modal dialog opens,

or operation, focusable components receive focus in an order that preserves meaning and operability. [5]	predictable, and preserves the meaning and operability of the content. This is vital for keyboard-only users and screen reader users. [80]	takes. Ensure it flows logically (e.g., top-to-bottom, left-to-right for LTR languages) and matches the visual layout where meaning is implied by order. Check that focus moves to opened dialogs/menus and returns appropriately when closed. **Tool-Assisted:** Some browser extensions can visualize tab order. **AT Testing:** Use a screen reader and navigate with Tab. Listen to the sequence of announced elements.	confusion (e.g., in a form). When a modal dialog opens, focus does not move into the dialog, or when it closes, focus returns to an illogical location. Focus skips over interactive elements. [80]	focus moves to the first interactive element within it. **Fail:** Tabbing through a login form jumps from "Username" to "Submit" then back to "Password."
2.4.4 Link Purpose (In Context) (Level A) The purpose of each link can be determined from the link text alone or from the link text together	To verify that the purpose of each link can be understood either from its text alone or from its text combined with its surrounding context (e.g., enclosing	**Manual:** Examine each link. Can its destination/purpose be understood from the link text itself? If not (e.g., "Read more"), is the purpose clear	Link text is generic (e.g., "Click here," "Read more," "More") without sufficient surrounding context. An image used as a link has no alt text or non-	**Pass:** Link text "Read more about our Q1 financial results" is self-explanatory. A "Read more" link is preceded by a news article summary: "Company X

with its programmatically determined link context, except where the purpose of the link would be ambiguous to users in general. [5]	sentence, list item, table cell, or ARIA attributes), enabling users (especially screen reader users who may navigate via a list of links) to decide whether to follow a link. [72]	from the immediate surrounding text (sentence, paragraph, list item, table cell/header) or from its aria-label or aria-labelledby attribute? [48] **Tool-Assisted:** Accessibility checkers can flag generic link text like "click here." Browser dev tools to check ARIA attributes. **AT Testing:** Use a screen reader to list all links on the page. Verify each link's purpose is clear from the announced text. Navigate to links and listen to how they are announced in context.	descriptive alt text. Multiple links on a page have the same link text but point to different destinations, and context doesn't differentiate them. [72]	announces new product. Read more." **Fail:** A page has five links all labeled "Download PDF" but each links to a different document, and there's no other context to distinguish them.
2.4.5 Multiple Ways (Level AA) More than one way is available to locate a web page within a set of web	To verify that users have at least two ways to find any given web page within a set of pages (e.g., site navigation +	**Manual:** Identify the primary navigation mechanisms (e.g., menus, breadcrumbs). Check for the presence of at	A website only provides a main navigation menu, with no search function or site map. Some content pages	**Pass:** A website offers a main navigation menu and a functional site search available on every page. **Fail:** A blog has

pages except where the Web page is the result of, or a step in, a process. [5]	search; site map + list of all pages), unless the page is part of a sequential process (like checkout or a survey). This accommodates different user preferences and navigation strategies. [81]	least one other site-wide mechanism like a site search, site map, or links to all pages from the homepage. For a representative sample of pages, confirm they can be reached by at least two methods. Verify exceptions for process steps (e.g., a checkout confirmation page). [81] **Tool-Assisted:** Site crawlers can help identify all pages and how they are linked, aiding in verifying if pages are reachable. **AT Testing:** Ensure all provided navigation methods are accessible.	are only reachable by following a specific, non-obvious sequence of links ("orphan pages"). A large site relies solely on a search function without any browseable navigation structure. [81]	many articles, but the only way to find older articles is to click "Previous Page" repeatedly; there is no search, archive, or category navigation.
2.4.6 Headings and Labels (Level AA) Headings and labels describe topic or	To verify that headings clearly describe the topic of the section they introduce, and that labels for	**Manual:** Read all headings on the page. Do they accurately reflect the content that follows? Read all	A section about "Contact Information" is headed "Details." A button that saves user data	**Pass:** A section discussing payment options is headed "Payment Methods." A

purpose. [5]	controls clearly describe their purpose. This helps users understand page structure and how to interact with components. (Note: This SC is about descriptiveness; programmatic markup is 1.3.1, presence of labels for inputs is 3.3.2). [82]	visible labels for form controls and other UI components. Do they clearly indicate the purpose or expected input? [48] **Tool - Assisted:** Tools can list headings, making it easier to review them for descriptiveness. **AT Testing:** Use a screen reader to navigate by headings. Listen to the announced headings and labels to ensure they are clear and make sense in context.	is labeled "Go." An input field for "Email Address" is labeled simply "Input." Headings are used for stylistic purposes only and don't describe distinct sections. [82]	button to add an item to a shopping cart is labeled "Add to Cart." **Fail:** A heading "Important!" is used for a section containing privacy policy updates, without mentioning privacy or policy.
2.4.7 Focus Visible (Level AA) Any keyboard operable user interface has a mode of operation where the keyboard focus indicator is visible. [5]	To verify that a visible focus indicator is present when interactive elements receive keyboard focus, allowing sighted keyboard users to discern their current location on the page. [83]	**Manual:** Navigate through all interactive elements using the keyboard (Tab, Shift+Tab, arrow keys). Visually confirm that each element that receives focus has a clear, visible indicator	CSS outline: none; is used on focusable elements without providing an alternative visible focus style. The focus indicator is too subtle (e.g., same color as the background or	**Pass:** When a link receives keyboard focus, a distinct border or outline appears around it. **Fail:** When tabbing to buttons, no visual change occurs to indicate which button currently

		(e.g., outline, border change, background change). Ensure this indicator is not removed by CSS (e.g., outline: none;).[84] **Tool-Assisted:** Browser developer tools can show focus states. Some automated tools can flag outline:none usage. **AT Testing:** Not directly for visibility, but ensures keyboard operability which relies on visible focus.	component border) to be easily discernible. Focus indicator is present but is obscured by other content (see also 2.4.11).[83]	has focus.
2.4.11 Focus Not Obscured (Minimum) (Level AA) When a user interface component receives keyboard focus, the component is not entirely hidden due to author-created content. (Note 1: Where content in a	To verify that when an interactive element receives keyboard focus, it is not completely hidden by other author-created content (e.g., sticky headers/footers, non-modal dialogs, cookie banners). At	**Manual:** Navigate through all focusable elements using the keyboard. Pay particular attention to elements near sticky/fixed headers or footers, or elements that might be covered by pop-ups or	A sticky footer completely covers a link or button when it receives keyboard focus. A non-modal cookie banner at the bottom of the screen entirely hides a focused element located behind it. A chat widget expands and	**Pass:** A sticky footer is present, but when an element behind it receives focus, the page scrolls slightly to ensure at least part of the focused element is visible above the footer. **Fail:** A "Subscribe to

configurable interface can be repositioned by the user, then only the initial positions of user-movable content are considered for testing and conformance of this success criterion. Note 2: Content opened by the user may obscure the component receiving focus. If the user can reveal the focused component without advancing the keyboard focus, the component with focus is not considered visually hidden due to author-created content.) [5]	least part of the focused component must remain visible. [5]	banners that don't trap focus. Verify that no focused element is *entirely* obscured. [11] **Tool-Assisted:** Browser developer tools to inspect element positions and z-index. **AT Testing:** While primarily visual, ensure that if content is obscured, screen readers still announce the correct element (though this is more 4.1.2).	completely covers a focused input field. [11]	Newsletter" sticky bar at the bottom of the screen completely hides a "Terms of Service" link when the link receives keyboard focus.
Guideline 2.5 Input Modalities	Make it easier for users to operate functionality through various inputs beyond keyboard.			
2.5.1 Pointer	To verify that	**Manual:** Identify	A map can only	**Pass:** An image

Gestures (Level A) All functionality that uses multipoint or path-based gestures for operation can be operated with a single pointer without a path-based gesture, unless a multipoint or path-based gesture is essential. (Note: This requirement applies to web content that interprets pointer actions (i.e., this does not apply to actions that are required to operate the user agent or assistive technology).) [5]	any functionality requiring complex pointer gestures (like pinch-to-zoom or swiping a specific path) also has an alternative that can be operated with a single pointer (e.g., a simple tap or click on a button), unless the complex gesture is essential for t he function. [87]	all functionalities that use multipoint (e.g., pinch zoom) or path-based (e.g., swipe, drag along a path) gestures. For each, check if an alternative single-pointer control (e.g., +/- buttons for zoom, arrow buttons for carousels) exists and performs the same function. Test on touch devices and with a mouse. [87] **Tool-Assisted:** Not directly applicable for identifying gesture alternatives. **AT Testing:** Users of alternative pointing devices (head pointers, etc.) benefit directly. Ensure single-pointer alternatives are themselves accessible.	be zoomed using a two-finger pinch gesture, with no on-screen zoom buttons. An image gallery can only be navigated by swiping images, with no next/previous buttons. A slider control requires precise dragging along its track, with no way to set th e value by tapping on the track or using input fields/buttons. [87]	carousel supports swipe navigation but also has visible "Next" and "Previous" buttons. A map allows pinch-to-zoom and also has "+" and "-" buttons for zooming. **Fail:** A drawing application requires users to draw a specific path -based gesture (e.g., a circle) to unlock a feature, with no alternative button.
2.5.2 Pointer	To verify that	**Manual:** For all	Clicking a	**Pass:** A button

Cancellation (Level A) For functionality that can be operated using a single pointer, at least one of the following is true: • **No Down-Event:** The down-event of the pointer is not used to execute any part of the function; • **Abort or Undo:** Completion of the function is on the up-event, and a mechanism is available to abort the function before completion or to undo the function after completion; • **Up Reversal:** The up-event reverses any outcome of the preceding down-event; • **Essential:** Completing the function on the down-event is essential. (Note: Functions that emulate a	for single pointer operations, activation occurs on the up-event (e.g., mouse release, finger lift) or provides a way to abort/undo if activated on the down-event, unless down-event activation is essential. This helps users avoid accidental activation if t hey press down on the wrong target. [13]	clickable/tappable elements: Press down (mouse button or finger) on an element. While still pressed, move the pointer/finger *off* the element. Release the pointer/finger. The action should *not* occur. Alternatively, if action occurs on down-event, check for an undo mechanism. Test drag-and-drop: can it be cancelled by dropping outside a valid target? [48] **Tool-Assisted:** Browser developer tools to inspect event listeners (mousedown vs click or mouseup). **AT Testing:** Not directly for pointer cancellation, but ensures predictability for users who might have imprecise	button triggers an action immediately on mousedown (mouse button press) without a way to cancel by dragging off before mouseup. Dragging an item and accidentally dropping it on a wrong target finalizes the move with no undo option. A long press on a touchscreen immediately triggers an irreversible action. [88]	activates only when the mouse button is released (up-event) while the pointer is still over the button. If a user drags an item and drops it in an invalid location, the item returns to its original position (abort). **Fail:** Tapping and holding a "Delete" button for 0.5 seconds immediately deletes an item without confirmation or an undo option, and the action triggers on the initial press (down-event).

keyboard or numeric keypad key press are considered essential.) [5]		pointer control.		
2.5.3 Label in Name (Level A) For user interface components with labels that include text or images of text, the name contains the text that is presented visually. (Note: A best practice is to have the text of the label at the start of the name.) [5]	To verify that the programmatic accessible name of a UI component includes its visible text label. This is crucial for speech input users (who speak the visible label to activate controls) and text-to-speech users (for consistency between visual and auditory information). [89]	**Manual:** Visually identify components with text labels (buttons, links, inputs with <label>). **Tool-Assisted:** Use browser developer tools (Accessibility pane) to inspect the "Accessible Name" of the component. Verify that the visible label text is part of this accessible name. **AT Testing:** Use speech input software to try activating controls by speaking their visible labels. Use a screen reader to listen to the announced name and compare it to the visible label. [89]	A button is visibly labeled "Search" but its aria-label is "Find articles," so speech input "Click Search" fails. A link visibly shows "Product Details" but its accessible name (from aria-label) is just "Details." An input field has a visible <label> "User Name" but its aria-label is "Login ID." [89]	**Pass:** A button with visible text "Submit Order" has an accessible name "Submit Order". A link with text "Read More" has an aria-label="Read More about Product X". **Fail:** A button visually labeled "Next Page" has an aria-label="Continue ".

2.5.4 Motion Actuation (Level A) Functionality that can be operated by device motion or user motion can also be operated by user interface components and responding to the motion can be disabled to prevent accidental actuation, except when: • **Supported Interface:** The motion is used to operate functionality through an accessibility supported interface; • **Essential:** The motion is essential for the function and doing so would invalidate the activity. [5]	To verify that any functionality triggered by device motion (e.g., shaking, tilting) or user motion (e.g., gestures via camera) can also be operated by standard UI controls (buttons, links) and that motion actuation can be disabled to prevent accidental triggering. This benefits users with motor impairments or mounted devices. [90]	**Manual:** Identify any features triggered by device/user motion. Verify that equivalent UI controls exist. Look for a setting to disable motion-based input. Test that disabling motion prevents motion triggers but UI controls still work. [90] **Tool-Assisted:** Not directly applicable. **AT Testing:** Not directly for motion, but ensure alternative UI controls are AT-accessible.	"Shake to undo" is the only way to undo an action, with no on-screen undo button. Tilting the device is the only way to scroll content. There is no option to disable motion-triggered features, leading to accidental activation for users with tremors. [90]	**Pass:** A "shake to shuffle playlist" feature also has an on-screen "Shuffle" button, and a setting exists to disable shake detection. **Fail:** A game requires tilting the device to steer, with no alternative on-screen controls, and tilt cannot be disabled.
2.5.7 Dragging Movements (Level AA) All functionality that uses a	To verify that any functionality requiring a dragging movement (e.g.,	**Manual:** Identify all drag-and-drop interfaces, sortable lists, sliders operated	A list can only be reordered by dragging items. A slider's value can only	**Pass:** A sortable list allows items to be reordered by drag-and-drop, but also

dragging movement for operation can be achieved by a single pointer without dragging, unless dragging is essential or the functionality is determined by the user agent and not modified by the author. (Note: This requirement applies to web content that interprets pointer actions (i.e., this does not apply to actions that are required to operate the user agent or assistive technology).) [5]	reordering a list, moving items on a canvas, operating a slider by dragging the thumb) can also be achieved using single pointer actions (taps or clicks) without dragging, unless dragging is essential. This helps users with motor impairments who find dragging difficult or impossible. [5]	by dragging, etc. For each, check if an alternative single-pointer method exists (e.g., "Move Up/Down" buttons for list items, input fields or +/- buttons for sliders, select-then-click-destination for drag-and-drop). [12] **Tool-Assisted:** Not directly applicable. **AT Testing:** Ensure single-pointer alternatives are AT-accessible.	be changed by dragging its thumb. Items on a task board can only be moved between columns by dragging. [12]	provides "Move Up" and "Move Down" buttons for each item. A map allows dragging to pan, but also has arrow buttons for panning. **Fail:** A file upload component only supports dragging files from the desktop, with no "Browse" button alternative.
2.5.8 Target Size (Minimum) (Level AA) The size of the target for pointer inputs is at least 24 by 24(https://www.w3.org/TR/WCAG22/#dfn-css-pixels), except when: • **Spacing:**	To verify that interactive targets (buttons, links, custom controls) are at least 24x24 CSS pixels in size, or if smaller, are adequately spaced from other targets, to prevent	**Manual:** Identify all pointer targets. **Tool-Assisted:** Use browser developer tools to measure the dimensions (width, height, padding) of targets. Use tools or visual aids (e.g., a	Icon buttons are 16x16 pixels with no additional padding and are placed close together. Small text links in a footer are less than 24px high and have minimal spacing between them. Controls in	**Pass:** All buttons on a page are at least 24x24 CSS pixels. A row of small (e.g., 20x20px) icon links have enough margin between them that 24px diameter circles centered on

Undersized targets (those less than 24 by 24 CSS pixels) are positioned so that if a 24 CSS pixel diameter circle is centered on the bounding box of each, the circles do not intersect another target or the circle for another undersized target; • **Equivalent:** The function can be achieved through a different control on the same page that meets this criterion; • **Inline:** The target is in a sentence or its size is otherwise constrained by the line-height of non-target text; • **User Agent Control:** The size of the target is determined by the user agent and is not modified by the author; • **Essential:** A particular	accidental activation by users with motor impairments or those using touchscreens. [10]	24px square overlay) to check size. For spacing exception, visually estimate or use tools to check if a 24px diameter circle around undersized targets would overlap. [91] **AT Testing:** Not directly for size/spacing, but small/close targets impact usability with alternative pointing devices.	a mobile interface are tiny and difficult to tap accurately. [13]	each would not overlap. **Fail:** A series of small pagination number links (e.g., 18x18px) are immediately adjacent to each other with no spacing.

presentation of the target is essential or is legally required for the information being conveyed. [5]				
Principle 3: Understandable				
Guideline 3.1 Readable	Make text content readable and understandable.			
3.1.1 Language of Page (Level A) The default human language of each web page can be programmatically determined. [5]	To verify that the primary language of each web page is programmatically declared (e.g., via the lang attribute on the <html> element), enabling browsers and assistive technologies to render text correctly and apply appropriate pronunciation rules. [93]	**Manual:** Use browser developer tools to inspect the <html> element for a lang attribute (e.g., <html lang="en">). Verify the language code is valid and matches the page's primary language. [48] **Tool-Assisted:** Automated accessibility checkers (Axe, WAVE) can detect missing or invalid lang	The <html> tag lacks a lang attribute. The lang attribute contains an invalid or incorrect language code (e.g., lang="english" instead of lang="en"). The declared language does not match the predominant language of the page content. [9]	**Pass:** A webpage primarily in English has <html lang="en">. **Fail:** A webpage written in French has <html lang="en"> or no lang attribute.

		attributes. **AT Testing:** Listen to a screen reader's pronunciation. Incorrect language declaration can lead to mispronunciation.		
3.1.2 Language of Parts (Level AA) The human language of each passage or phrase in the content can be programmatically determined except for proper names, technical terms, words of indeterminate language, and words or phrases that have become part of the vernacular of the immediately surrounding text. [5]	To verify that when a passage or phrase within a page is in a different language than the page's default language, this change is programmatically indicated (e.g., using a lang attribute on an appropriate HTML element). This allows AT to switch pronunciation rules. Exceptions exist for proper names, technical terms, etc. [94]	**Manual:** Identify any text passages or phrases in a different language from the page's primary language. Use browser developer tools to check if these sections are wrapped in an element with a lang attribute specifying the correct language (e.g., Bonjour). [48] **Tool-Assisted:** Some tools might identify foreign words but may not reliably check for correct lang attributes. 	A paragraph in English contains a sentence in Spanish, but the Spanish sentence is not marked with lang="es". A quotation in French is included on an English page without a lang="fr" attribute. [94]	**Pass:** An English article quotes a French phrase: He said, <q lang="fr">C'est la vie</q>. . **Fail:** An English page includes a list of city names from various countries, e.g., "Paris, Roma, Berlin," without marking "Roma" as Italian (lang="it") or "Berlin" as German (lang="de"), assuming they are not considered vernacular in the context.

		AT Testing: Use a screen reader. Listen for changes in pronunciation when encountering text marked in a different language.		
Guideline 3.2 Predictable	Make Web pages appear and operate in predictable ways.			
3.2.1 On Focus (Level A) When any user interface component receives focus, it does not initiate a change of context. [5]	To verify that merely setting focus on a UI component (e.g., tabbing to it) does not automatically trigger a significant change of context (e.g., submitting a form, opening a new window, moving focus elsewhere unexpectedly). Changes of context should generally occur upon activation, not just focus. [95]	**Manual:** Use the keyboard (Tab, Shift+Tab, arrow keys) to navigate to all interactive elements. Observe if any element receiving focus causes an automatic form submission, new window/tab opening, or unexpected shift of focus. [95] **Tool-Assisted:** Developer tools can help inspect event listeners (onfocus). **AT Testing:** Navigate with a	Tabbing to a radio button automatically submits the form. Focusing on a link opens it in a new window without user activation. Tabbing to an input field causes a help dialog to open and steal focus. [95]	**Pass:** Tabbing to a dropdown menu allows the user to then use arrow keys to select an option and Enter to activate, without any change of context occurring just from focusing on the dropdown or its options. **Fail:** As soon as a user tabs to a "Country" select list, the page reloads to display provinces for the default country without the user

		screen reader. Listen for unexpected page changes or announcements triggered solely by focus events.		explicitly activating the selection.
3.2.2 On Input (Level A) Changing the setting of any user interface component does not automatically cause a change of context unless the user has been advised of the behavior before using the component. [5]	To verify that changing the setting of a UI component (e.g., typing in a text field, selecting a radio button or checkbox, choosing an option in a select list) does not automatically cause a significant change of context, unless the user has been warned beforehand. [96]	**Manual:** Interact with form controls. Type text into input fields, select radio buttons/checkboxes, change options in select lists. Observe if any of these actions automatically trigger a form submission, new window, or unexpected focus shift. If such behavior occurs, check if there was prior advice to the user about this behavior. [96] **Tool-Assisted:** Developer tools to inspect event listeners (onchange, oninput). **AT Testing:** Perform input actions with a screen reader.	Selecting an option from a dropdown list automatically submits the form without a separate submit button or prior warning. Checking a checkbox immediately navigates to a new page. Typing text into a search field automatically triggers search results to load and focus to shift without user activation of a search button. [96]	**Pass:** A form has a "Sort by" dropdown. Changing the selection updates the sort order of a list on the *same page* without reloading or shifting focus significantly (this is a change of content, not context). A submit button is required to apply filters. **Fail:** Selecting a language from a <select> element immediately reloads the page in the new language without any warning.

		Listen for unexpected context changes.		
3.2.3 Consistent Navigation (Level AA) Navigational mechanisms that are repeated on multiple web pages within a set of web pages occur in the same relative order each time they are repeated, unless a change is initiated by the user. [5]	To verify that navigation elements (menus, links) repeated across multiple pages within a website appear in the same relative order each time. This aids predictability and helps users (especially those with cognitive or visual impairments) locate navigation items more easily. [97]	**Manual:** Identify blocks of navigation links/menus that are repeated on several pages within a defined set. Compare the order of these links/menu items across these pages. The order in the source code (DOM order) is what matters for "relative order." [48] **Tool-Assisted:** Not directly, but site crawlers or comparison tools might help identify differences in navigation blocks across pages. **AT Testing:** Screen reader users benefit greatly from consistent navigation. Test navigation with a screen reader across multiple	The main navigation menu has links in the order "Home, About, Services, Contact" on one page, but "Home, Services, About, Contact" on another page within the same site section. A "Search" button appears before the navigation links in the header on one page, but after them on another. [97]	**Pass:** A website's primary navigation menu (e.g., Home, Products, Support, Contact Us) appears in the exact same order at the top of every page. **Fail:** On the homepage, the footer links are "Privacy Policy

		pages.		
3.2.4 Consistent Identification (Level AA) Components that have the same functionality within a set of web pages are identified consistently. [5]	To verify that UI components (buttons, links, icons) that perform the same function across multiple pages are identified consistently (i.e., have consistent labels or accessible names). This helps users predict functionality and reduces cognitive load. [98]	**Manual:** Identify components with the same functionality repeated across a set of pages (e.g., a "Search" icon/button, a "Print" icon, "Next Page" link). Check their visible labels and accessible names (using dev tools) for consistency. "Consistent" doesn't always mean "identical" if context makes the function clear (e.g., "Download PDF Report" vs. "Download PDF Invoice" for different documents is consistent). [99] **Tool-Assisted:** Not directly, but can help inspect labels and accessible names. **AT Testing:** Use a screen reader. Listen to how identically	A magnifying glass icon is used for "Search" on one page (alt="Search") but for "View Details" on another page (alt="View Details"), even if both lead to a search function. A button labeled "Submit" on one form and "Send" on another form, where both perform the same submission action. Two visually identical "Save" icons have different aria-labels on different pages. [99]	**Pass:** An icon of a shopping cart is used on multiple pages. Its accessible name is consistently "Shopping Cart" or "View Cart." **Fail:** A "Next" arrow icon is labeled "Continue to Step 2" on one page of a wizard, but the same icon is labeled "Proceed" on the next step for the same function of advancing.

		functioning components are announced on different pages. Inconsistent announcements can confuse users.		
3.2.6 Consistent Help (Level A) If a web page contains any of the following help mechanisms, and those mechanisms are repeated on multiple web pages within a set of web pages, they occur in the same order relative to other page content, unless a change is initiated by the user: • Human contact details; • Human contact mechanism; • Self-help option; • A fully automated contact mechanism. [5]	To verify that if help mechanisms (like contact info, FAQs, chat widgets) are repeated on multiple pages within a set, they appear in the same relative order in the page structure. This makes it easier for users, especially those with cognitive disabilities, to find help when needed. [5]	**Manual:** Identify any help mechanisms (contact links, FAQ links, chat widgets, phone numbers) repeated across a set of pages. Check their order in the HTML source relative to other consistent page elements (e.g., always after main navigation, always before footer) . [14] **Tool-Assisted:** Not directly applicable for checking relative order of specific help items. **AT Testing:** Consistent placement helps AT users predict where to find help links/controls.	A "Live Chat" link is in the header on the homepage but in the footer on product pages. A "Help" link is the first item in the footer on one page but the last item in the footer on another. Contact phone number is in the header on some pages and in a sidebar on others. [14]	**Pass:** A "Support" link is consistently the last item in the main navigation menu on all pages of a website. A "FAQ" link is always present in the page footer, appearing after copyright information and before social media links. **Fail:** On a multi-page application form, the "Help with this form" link is in the right sidebar on step 1, but at the bottom of the main content on step 2.

Guideline 3.3 Input Assistance	Help users avoid and correct mistakes.			
3.3.1 Error Identification (Level A) If an input error is automatically detected, the item that is in error is identified and the error is described to the user in text. [5]	To verify that when an input error is detected (e.g., a required field is empty, data format is incorrect), the error is clearly identified to the user in text, specifying which field is in error and what the error is. [100]	**Manual:** Intentionally make errors when filling out forms (leave required fields blank, enter invalid data types/formats). Submit the form. Check if error messages appear, are in text, clearly identify the field(s) in error, and describe the error. [48] **Tool-Assisted:** Some tools can check for programmatic association of error messages with form fields (e.g., using aria-describedby). **AT Testing:** Use a screen reader. Submit a form with errors. Verify that the screen reader announces the error messages and associates them with the	Form submission fails, but no error message is displayed, or errors are only indicated by color (e.g., red border) without text. A generic error message like "Invalid input" is shown without specifying which field or what was wrong. Error messages are not programmatically associated with their respective input fields. [100]	**Pass:** Upon submitting a form with an empty required field, a message "Username is required." appears next to the username field, and the field itself is highlighted. **Fail:** A form simply reloads after an error, with no text indicating what went wrong or which field caused the error.

		correct form fields.		
3.3.2 Labels or Instructions (Level A) Labels or instructions are provided when content requires user input. [5]	To verify that all content requiring user input (e.g., form fields) has clear labels or instructions to guide the user on what input is expected. This benefits all users, especially those with cognitive disabilities or screen reader users. [101]	**Manual:** Visually inspect all form fields and other input areas. Check that each has a visible text label clearly associated with it, or that clear instructions are provided for the input. For groups of controls (e.g., radio buttons for a question), ensure the group has a label (e.g., via <legend>). [102] **Tool-Assisted:** Accessibility checkers can flag inputs missing associated <label> elements. **AT Testing:** Use a screen reader to navigate to each input field. Verify that a descriptive label or instruction is announced for each.	Input fields are present without any visible text labels. Labels are visually near inputs but not programmatically linked (e.g., no for/id pairing, no aria-labelledby). Placeholder text is used as the *only* label (placeholder text disappears on input). Instructions for complex data formats are missing. [101]	**Pass:** Each text input field has a corresponding <label> element correctly associated with it using for and id attributes. A complex date format requirement is explained in text next to the date input fields. **Fail:** A search input field only contains placeholder text "Search here..." with no visible persistent label.

3.3.3 Error Suggestion (Level AA) If an input error is automatically detected and suggestions for correction are known, then the suggestions are provided to the user, unless it would jeopardize the security or purpose of the content. [5]	To verify that when input errors are detected and correction suggestions are known (e.g., mistyped username, incorrect date format), these suggestions are provided to the user to help them fix the error. This benefits users with learning disabilities or those who have difficulty understanding how to correct errors. [98]	**Manual:** Intentionally make common input errors for which suggestions are feasible (e.g., misspell a common word in a field with spellcheck, enter a date in DD/MM/YYYY when MM/DD/YYYY is expected). Submit the form. Check if error messages not only identify the error but also provide a suggestion for correction. **Tool-Assisted:** Not directly for suggestion quality, but can help identify error states. **AT Testing:** Use a screen reader. Verify that error suggestions are announced clearly along with the error identification.	An error message says "Invalid date" but doesn't suggest the correct format (e.g., "Please use MM/DD/YYYY format"). A username field indicates "Username taken" but offers no suggestions for alternatives or indicates which part is problematic. Suggestions are provided but are unclear or unhelpful. [98]	**Pass:** If a user types "Decmber" in a month field, the error message says "Invalid month. Did you mean December?" **Fail:** A field requiring a selection from "Option A, Option B, Option C" shows an error "Invalid selection" if the user types "Option D", without listing the valid options as a suggestion.
3.3.4 Error Prevention (Legal,	To verify that for critical actions (legal	**Manual:** Identify any pages/processe	A "Confirm Purchase" button executes	**Pass:** An e-commerce checkout

Financial, Data) (Level AA) For web pages that cause legal commitments or financial transactions for the user to occur, that modify or delete user-controllable data in data storage systems, or that submit user test responses, at least one of the following is true: • **Reversible:** Submissions are reversible. • **Checked:** Data entered by the user is checked for input errors and the user is provided an opportunity to correct them. • **Confirmed:** A mechanism is available for reviewing, confirming, and correcting information before finalizing the submission.	commitments, financial transactions, data deletion/modification, test submissions), users can reverse the action, check and correct data before submission, or review and confirm information before finalizing. This helps prevent serious, irreversible errors. [104]	s involving legal commitments (e.g., agreeing to terms), financial transactions (e.g., purchases), data modification/deletion (e.g., deleting user profile), or test submissions. For each, verify that at least one of the three conditions is met: 1. Reversible: Can the action be undone after submission (e.g., cancel order within X minutes)? 2. Checked: Does the system validate input and allow corrections before final submission? (overlaps with 3.3.1, 3.3.3) 3. Confirmed: Is there a final review step where all data can be checked and corrected before committing? [104] **Tool -**	a non-refundable transaction immediately without a prior review screen. Deleting a user account happens instantly upon clicking "Delete Account" with no confirmation step or undo option. Submitting an online exam is final with no opportunity to review answers before submission. [104]	process includes an "Order Review" page showing all items, shipping, and payment details, with an "Edit" button for each section, before the final "Place Order" button. **Fail:** A system allows users to modify critical settings, and clicking "Save" applies changes immediately without a confirmation dialog or an option to revert to previous settings.

[5]		**Assisted:** Not directly applicable. **AT Testing:** Ensure all confirmation, correction, and reversal mechanisms are fully accessible.		
3.3.7 Redundant Entry (Level A) Information previously entered by or provided to the user that is required to be entered again in the same process is either: • auto-populated, or • available for the user to select. Except when: • re-entering the information is essential, • the information is required to ensure the security of the content, or • previously entered information is no longer valid. [5]	To verify that in a multi-step process within the same session, information already provided by the user is either auto-populated or made available for selection if required again. This reduces cognitive load and effort, especially for users with memory or motor impairments. [5]	**Manual:** Identify multi-step forms or processes. Note information entered in early steps. In later steps, if the same information is requested, check if it's auto-populated or if there's a mechanism to select it (e.g., a dropdown, a "use previous address" checkbox). Test exceptions (e.g., password confirmation fields, re-entering expired credit card). [15] **Tool-Assisted:** Not directly applicable. **AT Testing:** Ensure auto-	In a checkout process, after entering shipping address, the billing address fields are blank and require full re-entry, even if they are the same. A multi-page survey asks for the user's email on page 1 and again on page 5, with no auto-fill or option to recall. [15]	**Pass:** In a flight booking, after selecting departure and return dates on step 1, these dates are pre-filled or clearly displayed for confirmation on the review step. A checkout form has a checkbox "Billing address same as shipping address" which, when checked, auto-populates billing fields. **Fail:** A user enters their name and address in step 1 of a registration process. In step 3, for account verification, they are asked to re-type their full name and

		population or selection mechanisms are AT-accessible.		address from memory, with no fields pre-filled.

| **3.3.8 Accessible Authentication (Minimum) (Level AA)**
 A cognitive function test (such as remembering a password or solving a puzzle) is not required for any step in an authentication process unless that step provides at least one of the following:
 • **Alternative:** Another authentication method that does not rely on a cognitive function test.
 • **Mechanism:** A mechanism is available to assist the user in completing the cognitive function test.
 • **Object Recognition:** The cognitive function test is to recognize objects.
 • **Personal Content:** The cognitive function test is to identify [non-text content](https://www.w3.org/TR/WCAG22/#dfn-non

Works cited

1. 5 Common WCAG Mistakes and How to Avoid Them - BarrierBreak, accessed on June 4, 2025, https://www.barrierbreak.com/5-common-wcag-mistakes-and-how-to-avoid-them/
2. What is Accessibility Testing: Examples, Types, Metrics | BrowserStack, accessed on June 4, 2025, https://www.browserstack.com/accessibility-testing/what-is-accessibility-testing
3. 7 Essential Steps In Your Accessibility Testing Checklist For Unbeatable Website Compliance - BetterQA, accessed on June 4, 2025, https://betterqa.co/blog/web-accessibility-testing-checklist/
4. How to Integrate Ethical and Accessibility Considerations into QA Processes - TestDevLab, accessed on June 4, 2025, https://www.testdevlab.com/blog/ethical-and-accessibility-considerations-in-qa
5. Web Content Accessibility Guidelines (WCAG) 2.2 - W3C, accessed on June 4, 2025, https://www.w3.org/TR/WCAG22/
6. WCAG 2 Overview | Web Accessibility Initiative (WAI) | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/standards-guidelines/wcag/
7. WCAG 2.2 Compliance What is it? | Principles & Guidelines - UserWay, accessed on June 4, 2025, https://userway.org/compliance/wcag/
8. WCAG Compliance | Guide | AudioEye®, accessed on June 4, 2025, https://www.audioeye.com/compliance/wcag/
9. The 6 most common WCAG failures and how to fix them - Recite Me, accessed on June 4, 2025, https://reciteme.com/us/news/6-most-common-wcag-failures/
10. What's New in 2.2?- Digital Accessibility Office - Carnegie Mellon University, accessed on June 4, 2025, https://www.cmu.edu/computing/dao/news/2023/nov/wcag-2.2.html

11. Understanding Success Criterion 2.4.11: Focus Not Obscured ...- W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/focus-not-obscured-minimum.html
12. Understanding Success Criterion 2.5.7: Dragging Movements | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/dragging-movements.html
13. Understanding Success Criterion 2.5.8: Target Size (Minimum) | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html
14. Understanding Success Criterion 3.2.6: Consistent Help | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/consistent-help.html
15. Understanding Success Criterion 3.3.7: Redundant Entry | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/redundant-entry.html
16. Understanding Success Criterion 3.3.8: Accessible Authentication ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/accessible-authentication-minimum.html
17. Understanding WCAG 2.2 | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/
18. QA Tester - DWP Accessibility Manual, accessed on June 4, 2025, https://accessibility-manual.dwp.gov.uk/guidance-for-your-job-role/qa-tester
19. Writing effective Accessibility Bug Reports - Accessibility Consulting - Netz-barrierefrei.de, accessed on June 4, 2025, https://netz-barrierefrei.de/en/bug-reports.html
20. Template for Reporting Accessibility Issues - Harvard's Digital Accessibility, accessed on June 4, 2025, https://accessibility.huit.harvard.edu/template-reporting-accessibility-issues
21. 7 tips for devs and designers to set your accessibility efforts up for success | Deque, accessed on June 4, 2025, https://www.deque.com/blog/7-tips-for-devs-and-designers-to-set-your-accessibility-efforts-up-for-success/
22. How to Use Assistive Technology to Detect Accessibility Issues - TestDevLab, accessed on June 4, 2025, https://www.testdevlab.com/blog/accessibility-testing-using-assistive-technologies
23. The Best AI Accessibility Testing Tools You Shouldn't Miss in 2025, accessed on June 4, 2025, http://www.captioningstar.com/blog/ai-accessibility-testing-tools-2025/
24. How to Shift Left with Accessibility in Software Development - AEL Data, accessed on June 4, 2025, https://aeldata.com/how-to-shift-left-with-accessibility/
25. QA in Agile Methodology: Best Practices & Implementation for 2025 - ACCELQ, accessed on June 4, 2025, https://www.accelq.com/blog/quality-assurance-in-agile-methodology/

26. Understanding Success Criterion 1.1.1: Non-text Content | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/non-text-content.html
27. Accessibility Testing Tools & Software: Axe - Deque Systems, accessed on June 4, 2025, https://www.deque.com/axe/
28. WCAG Explained: Complete Guide for Your Accessibility Testing - TestDevLab, accessed on June 4, 2025, https://www.testdevlab.com/blog/wcag-explained-complete-guide-for-your-accessibility-testing
29. Screen Reader Testing: What it is & How to Test Them - QA Touch, accessed on June 4, 2025, https://www.qatouch.com/blog/screen-reader-testing/
30. How to Test Websites with Screen Readers | BrowserStack, accessed on June 4, 2025, https://www.browserstack.com/guide/test-websites-with-screen-readers
31. Five Most Common Accessibility Errors in Software Design and Development - WCAG, accessed on June 4, 2025, https://www.wcag.com/blog/five-most-common-accessibility-errors-in-software-design-and-development/
32. Understanding Success Criterion 1.2.1: Audio-only and Video-only ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/audio-only-and-video-only-prerecorded.html
33. Understanding Success Criterion 1.2.1: Audio-only and Video-only (Prerecorded) - W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG21/Understanding/audio-only-and-video-only-prerecorded.html
34. WCAG Audio-only and Video-only (Prerecorded) Explained - Stark, accessed on June 4, 2025, https://www.getstark.co/wcag-explained/perceivable/time-based-media/audio-only-and-video-only-prerecorded/
35. Understanding Success Criterion 1.2.2: Captions (Prerecorded ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/captions-prerecorded.html
36. Testing Methods - Pre-recorded Captions - Dennis Deacon, accessed on June 4, 2025, https://www.dennisdeacon.com/web/accessibility/testing-methods-pre-recorded-captions/
37. Understanding Success Criterion 1.2.2: Captions (Prerecorded) | WAI - W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG21/Understanding/captions-prerecorded.html
38. Understanding Success Criterion 1.2.3: Audio Description or Media ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/audio-description-or-media-alternative-prerecorded.html
39. Understanding Success Criterion 1.2.3: Audio Description or Media Alternative (Prerecorded) | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG21/Understanding/audio-description-or-media-alternative-prerecorded.html
40. Understanding Success Criterion 1.2.3 | Understanding WCAG 2.0 - W3C, accessed on June 4, 2025, https://www.w3.org/TR/UNDERSTANDING

WCAG20/media-equiv-audio-desc.html
41. Understanding Success Criterion 1.2.4: Captions (Live) | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/captions-live.html
42. Understanding Success Criterion 1.2.5: Audio Description ... - W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/audio-description-prerecorded.html
43. Understanding Success Criterion 1.3.1: Info and Relationships | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/info-and-relationships.html
44. Using WCAG Success Criteria 1.3.1 to Improve Accessibility - AudioEye, accessed on June 4, 2025, https://www.audioeye.com/post/using-wcag-131-to-improve-accessibility/
45. Understanding Success Criterion 1.3.1: Info and Relationships | WAI - W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG21/Understanding/info-and-relationships.html
46. Understanding Success Criterion 1.3.2: Meaningful Sequence | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/meaningful-sequence.html
47. Understanding Success Criterion 1.3.3: Sensory Characteristics ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/sensory-characteristics.html
48. WCAG Checklist: A Simplified Guide to WCAG 2.2 AA - DigitalA11Y, accessed on June 4, 2025, https://www.digitala11y.com/wcag-checklist/
49. Testing - Accessibility - NHS digital service manual, accessed on June 4, 2025, https://service-manual.nhs.uk/accessibility/testing
50. Understanding Success Criterion 1.3.4: Orientation | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/orientation.html
51. Understanding Success Criterion 1.3.5: Identify Input Purpose | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/identify-input-purpose.html
52. Understanding Success Criterion 1.4.1: Use of Color | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/use-of-color.html
53. 1.4.1 Use of Color (Level A) - WCAG, accessed on June 4, 2025, https://www.wcag.com/designers/1-4-1-use-of-color/
54. WCAG 2.1 - SC 1.4.1 Use of Color - Bureau of Internet Accessibility, accessed on June 4, 2025, https://www.boia.org/wcag2/cp/1.4.1
55. Understanding Success Criterion 1.4.2: Audio Control | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/audio-control.html
56. Understanding Success Criterion 1.4.3: Contrast (Minimum) | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/contrast-

minimum.html
57. The 15 Best Online Contrast Checker Tools in 2025 - accessiBe, accessed on June 4, 2025, https://accessibe.com/blog/knowledgebase/color-contrast-checker-tools
58. Colour Contrast Analyser (CCA) - TPGi, accessed on June 4, 2025, https://www.tpgi.com/color-contrast-checker/
59. Understanding Success Criterion 1.4.13: Content on Hover or Focus ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/content-on-hover-or-focus.html
60. Understanding Success Criterion 1.4.4: Resize Text | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/resize-text.html
61. Understanding Success Criterion 1.4.5: Images of Text | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/images-of-text.html
62. Understanding Success Criterion 1.4.10: Reflow | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/reflow.html
63. Understanding Success Criterion 1.4.11: Non-text Contrast | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/non-text-contrast.html
64. Understanding Success Criterion 1.4.12: Text Spacing | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/text-spacing.html
65. Understanding Success Criterion 2.2.2: Pause, Stop, Hide | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/pause-stop-hide.html
66. Understanding Success Criterion 2.1.1: Keyboard | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/keyboard.html
67. WCAG 2.1.1 Keyboard Accessibility Explained - UXPin, accessed on June 4, 2025, https://www.uxpin.com/studio/blog/wcag-211-keyboard-accessibility-explained/
68. Web Content Accessibility Guidelines (WCAG) 2.1 - W3C, accessed on June 4, 2025, https://www.w3.org/TR/WCAG21/
69. Understanding Success Criterion 2.1.2: No Keyboard Trap | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/no-keyboard-trap.html
70. Understanding Success Criterion 2.1.4: Character Key Shortcuts ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/character-key-shortcuts.html
71. Understanding Success Criterion 2.2.1: Timing Adjustable | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/timing-adjustable.html
72. Understanding Success Criterion 2.4.4: Link Purpose (In Context ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/link-purpose-in-context.html

73. Understanding Success Criterion 2.3.1: Three Flashes or Below ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/three-flashes-or-below-threshold.html
74. Success Criterion 2.3.1 Three Flashes or Below Threshold - Equally.AI Blog, accessed on June 4, 2025, https://blog.equally.ai/developer-guide/2-3-1-three-flashes-or-below-threshold/
75. WCAG 2.3.1: Three flashes or below threshold (Level A) - Silktide, accessed on June 4, 2025, https://silktide.com/accessibility-guide/the-wcag-standard/2-3/seizures-and-physical-reactions/2-3-1-three-flashes-or-below-threshold/
76. Understanding Success Criterion 2.4.1: Bypass Blocks | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/bypass-blocks.html
77. Understanding Success Criterion 2.4.2: Page Titled | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/page-titled.html
78. Understanding Success Criterion 2.4.2 | Understanding WCAG 2.0 - W3C, accessed on June 4, 2025, https://www.w3.org/TR/UNDERSTANDING-WCAG20/navigation-mechanisms-title.html
79. WCAG 2.1 - SC 2.4.2 Page Titled - Bureau of Internet Accessibility, accessed on June 4, 2025, https://www.boia.org/wcag2/cp/2.4.2
80. Understanding Success Criterion 2.4.3: Focus Order | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/focus-order.html
81. Understanding Success Criterion 2.4.5: Multiple Ways | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/multiple-ways.html
82. Understanding Success Criterion 2.4.6: Headings and Labels | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/headings-and-labels.html
83. Understanding Success Criterion 2.4.7: Focus Visible | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/focus-visible.html
84. WebAIM's WCAG 2 Checklist, accessed on June 4, 2025, https://webaim.org/standards/wcag/checklist
85. Web Accessibility Guidelines - WCAG 2 Checklist - Clym, accessed on June 4, 2025, https://clym.io/accessibility-news/wcag-2-checklist
86. WCAG 2.2: New Success Criteria, More Inclusive Content, accessed on June 4, 2025, https://www.wcag.com/blog/wcag-2-2-aa-summary-and-checklist-for-website-owners/
87. Understanding Success Criterion 2.5.1: Pointer Gestures | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/pointer-gestures.html
88. Understanding Success Criterion 2.5.2: Pointer Cancellation | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/pointer-

cancellation.html
89. Understanding Success Criterion 2.5.3: Label in Name | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/label-in-name.html
90. Understanding Success Criterion 2.5.4: Motion Actuation | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/motion-actuation.html
91. WCAG 2.2. Success Criteria 2.5.8, Target Size (Level AA) - A Summary, accessed on June 4, 2025, https://jimbyrne.co.uk/wcag-2-2-success-criteria-2-5-8-target-size-a-summary/
92. How to test 2.5.8 Target Size (Minimum) - TPGi, accessed on June 4, 2025, https://www.tpgi.com/how-to-test-2-5-8-target-size-minimum/
93. Understanding Success Criterion 3.1.1: Language of Page | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/language-of-page.html
94. Understanding Success Criterion 3.1.2: Language of Parts | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/language-of-parts.html
95. Understanding Success Criterion 3.2.1: On Focus | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/on-focus.html
96. Understanding Success Criterion 3.2.2: On Input | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/on-input.html
97. Understanding Success Criterion 3.2.3: Consistent Navigation | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/consistent-navigation.html
98. Understanding Success Criterion 3.3.3: Error Suggestion | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/error-suggestion.html
99. Understanding Success Criterion 3.2.4: Consistent Identification ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/consistent-identification.html
100. Understanding Success Criterion 3.3.1: Error Identification | WAI | W3C, accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/error-identification.html
101. Understanding Success Criterion 3.3.2: Labels or Instructions | WAI ..., accessed on June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/labels-or-instructions.html
102. Understanding WCAG 3.3.2 Labels or Instructions, accessed on June 4, 2025, https://www.wcag.com/developers/3-3-2-labels-instructions/
103. Guide to Accessible Web Design & Development - Section508.gov, accessed on June 4, 2025, https://www.section508.gov/develop/guide-accessible-web-design-development/
104. Understanding Success Criterion 3.3.4: Error Prevention (Legal ..., accessed on

June 4, 2025, https://www.w3.org/WAI/WCAG22/Understanding/error-prevention-legal-financial-data.html
105. WCAG 2.2 At A Glance - 3.3.7 Redundant Entry (Level A) - YouTube, accessed on June 4, 2025, https://www.youtube.com/watch?v=jKirEdlHMao